

Version 1: A* Path Planning on a Heightmap

We consider a 2D grid derived from a grayscale heightmap, where each cell stores an elevation value $H(y, x)$ in meters. A path is a sequence of connected cells from a start (y_s, x_s) to a goal (y_g, x_g) .

Heuristic

The heuristic is the straight-line Euclidean distance in meters from a cell (y, x) to the goal:

$$h(y, x) = \sqrt{(y_g - y)^2 + (x_g - x)^2} \cdot c,$$

where c is the cell size in meters (`cell_size_m`).

Slope and Roughness

For each cell, the terrain slope is estimated using gradients of the elevation field:

$$\frac{\partial H}{\partial x}, \quad \frac{\partial H}{\partial y},$$
$$\theta(y, x) = \arctan\left(\sqrt{\left(\frac{\partial H}{\partial x}\right)^2 + \left(\frac{\partial H}{\partial y}\right)^2}\right),$$

where $\theta(y, x)$ is the slope angle in radians. The slope in degrees is

$$\theta_{\text{deg}}(y, x) = \frac{180}{\pi} \theta(y, x).$$

Roughness is approximated as the local standard deviation of elevation in a window of size $w \times w$:

$$r(y, x) = \sqrt{\mathbb{E}[H^2] - (\mathbb{E}[H])^2}.$$

Step Cost

Moving from a cell (y_0, x_0) to a neighbor (y_1, x_1) has a base distance cost

$$d = m \cdot c,$$

where $m = 1$ for cardinal moves and $m = \sqrt{2}$ for diagonal moves.

The elevation change is

$$\Delta z = H(y_1, x_1) - H(y_0, x_0),$$

with uphill and downhill components

$$\Delta z^+ = \max(0, \Delta z), \quad \Delta z^- = \max(0, -\Delta z).$$

The full step cost is then

$$\text{cost}(y_0, x_0 \rightarrow y_1, x_1) = d + w_{\text{up}} \cdot \Delta z^+ + w_{\text{down}} \cdot \Delta z^- + w_{\text{slope}} \cdot \frac{\theta_{\text{deg}}(y_1, x_1)}{\theta_{\text{max}}} + w_{\text{rough}} \cdot r(y_1, x_1),$$

where

- w_{up} = uphill penalty weight,
- w_{down} = downhill penalty weight,
- w_{slope} = slope penalty weight,
- w_{rough} = roughness penalty weight,
- θ_{max} = maximum allowed slope in degrees.

Cells with $\theta_{\text{deg}} > \theta_{\text{max}}$ are considered non-traversable (cost = ∞).

A* Update Rule

The A* algorithm maintains for each cell a tentative cost-to-come $g(y, x)$ and computes the evaluation function

$$f(y, x) = g(y, x) + h(y, x).$$

At each step, the cell with minimum f is expanded until the goal is reached or the search fails.